

MODULE II • LECTURE 06

Advanced CNNs for Text Classification

Padding • Stride • Multiple Filters • Multi-Channel Inputs • Architecture Design

Dr Tamal Ghosh

Module II Progress

L1: Feedforward Neural Networks



L2: Activation Functions



L3: Dropout & Network Design



L4: Backpropagation & Optimization



L5: CNNs for Text Classification



L6: Advanced CNN Architecture

The Pipeline

Text

- Tokenization
- Word Embeddings
- Sentence Matrix
- Convolution
- ReLU
- Max Pooling
- Concatenation
- Fully Connected Layer
- Classification

Key Idea

Learn local patterns automatically instead of manually defining n-grams.



Why Go Beyond the Basic CNN?

A simple CNN works well, but real NLP problems introduce challenges:

- > Sentences have different lengths.
- > Important patterns may occur at different positions.
- > Different patterns have different sizes.
- > One filter cannot capture every linguistic pattern.
- > Boundary words may contain useful information.
- > Architecture choices significantly affect performance.

Therefore: CNN design matters.

The Sentence Matrix

Suppose:

- > Sentence length = 6
- > Embedding dimension = 4

Then the matrix is X

$$X \in \mathbb{R}^{6 \times 4}$$

Word	Embedding (d=4)
The	[0.2, 0.1, 0.4, 0.3]
movie	[0.7, 0.5, 0.2, 0.6]
was	[0.1, 0.3, 0.4, 0.2]
extremely	[0.8, 0.4, 0.7, 0.5]
good	[0.9, 0.8, 0.6, 0.7]
today	[0.2, 0.3, 0.1, 0.4]

The Filter

Filter Dimensions

Consider a filter covering 3 words.

If embedding dimension = : d

$$W \in \mathbb{R}^{3 \times d}$$

The filter examines three consecutive words at a time.

Sliding Windows

```
[The movie was]
[movie was extremely]
[was extremely good]
[extremely good today]
```

Each window produces one feature.

Padding: The Boundary Problem

What happens at the boundaries of the sentence?



Without Padding

[The movie was]

[movie was extremely]

[was extremely good]

[extremely good today]

The filter never explicitly sees "The" or "today" in the context of being a boundary word.



With Padding

Padding adds special tokens to the sequence:

The movie was ...

This allows boundary positions to participate fully in the convolution process.

Valid vs Same Padding

"Valid" Padding

No padding is applied.

The output sequence becomes shorter.

$$L_{\text{out}} = L - K + 1$$

NLP Intuition: Focus only on complete, existing word windows.

"Same" Padding

Padding is added so that output length is approximately preserved.

For stride 1:

$$L_{\text{out}} \approx L$$

NLP Intuition: Preserve positional coverage across the entire sentence length.

Padding Example

L=5

Input Length

K=3

Kernel Size

Valid Padding Output

$$5 - 3 + 1 = 3$$

[1 2 3]

[2 3 4]

[3 4 5]

Same Padding Output

Padding allows approximately 5 output positions to be calculated, maintaining the sequence structure.

Stride

Stride = how far the filter moves at each step.



Stride 1

```
[1 2 3]
 [2 3 4]
  [3 4 5]
```

Moves one word at a time. Dense feature extraction.



Stride 2

```
[1 2 3]
      [3 4 5]
```

Skips words. Larger stride reduces output size and computation, but may lose fine-grained information.



Convolution Output Size

A fundamental formula for network architecture design.

$$L_{\text{out}} = \frac{L + 2P - K}{S} + 1$$

L = input length

P = padding

K = kernel size

S = stride

Important: This formula is worth remembering.

Numerical Example

Given Parameters

L = 10 (sequence length)

K = 3 (filter size)

P = 1 (padding on each side)

S = 1 (stride)

Calculation

$$L_{\text{out}} = \frac{10 + 2(1) - 3}{1} + 1$$

= 10

Therefore:

Same padding + stride 1 → same sequence length.

Multiple Filters

One filter learns one type of pattern.

But language contains many patterns. So we use:

$$W_1, W_2, W_3, \dots, W_m$$

Each filter can learn a different feature simultaneously.

- > **Filter 1** → Negation
- > **Filter 2** → Positive sentiment
- > **Filter 3** → Negative sentiment
- > **Filter 4** → Intensifier
- > **Filter 5** → Domain-specific phrase

Multiple Feature Maps



Convolution

Suppose we use 100 filters.

Each filter produces one feature map.

100 filters $\Rightarrow$ 100 feature maps



Pooling & Classification

After global max pooling on each map:

100 feature maps $\Rightarrow$ 100 features

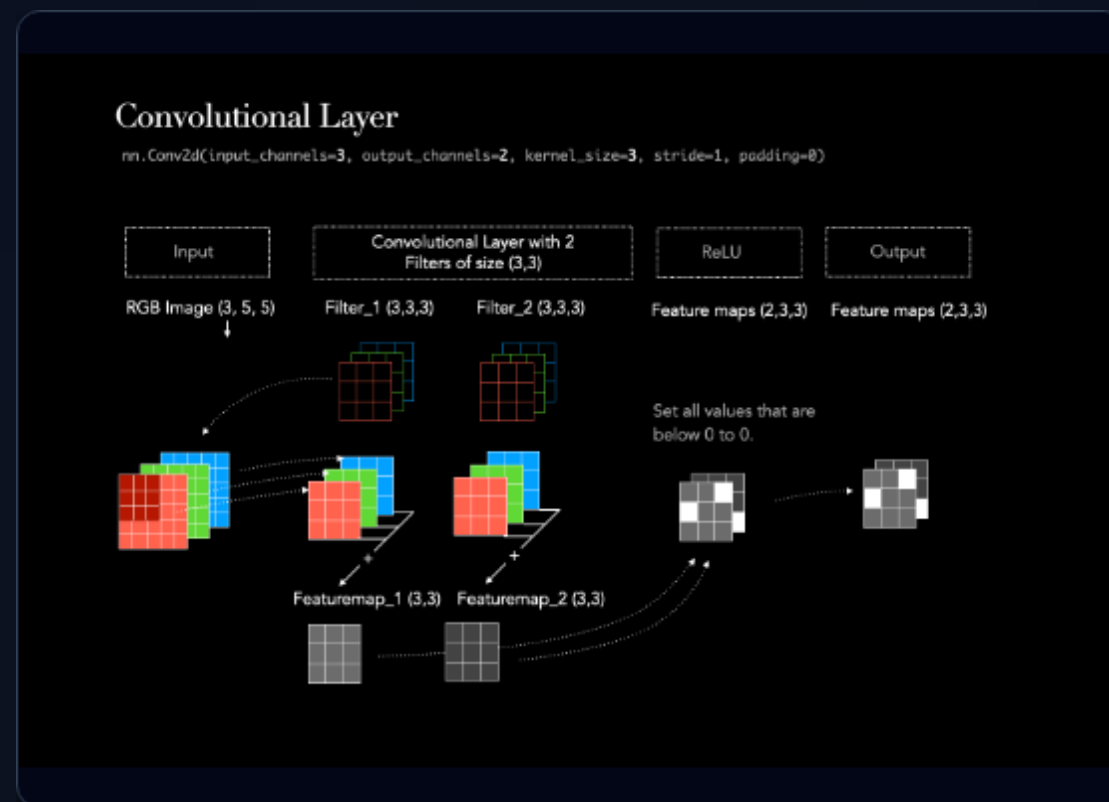
These 100 values become the input vector to the classifier.

Different Kernel Sizes

Language patterns occur at different lengths.

- > **Kernel = 2:** Captures short patterns.
(e.g., "not good")
- > **Kernel = 3:** Captures phrase-level patterns.
(e.g., "not very good")
- > **Kernel = 5:** Captures longer local patterns.
(e.g., "not at all very good")

Different kernels can capture different structures simultaneously.



Multi-Kernel CNN Architecture

A common architecture uses parallel convolution layers.



Key Idea: Use multiple receptive-field sizes.

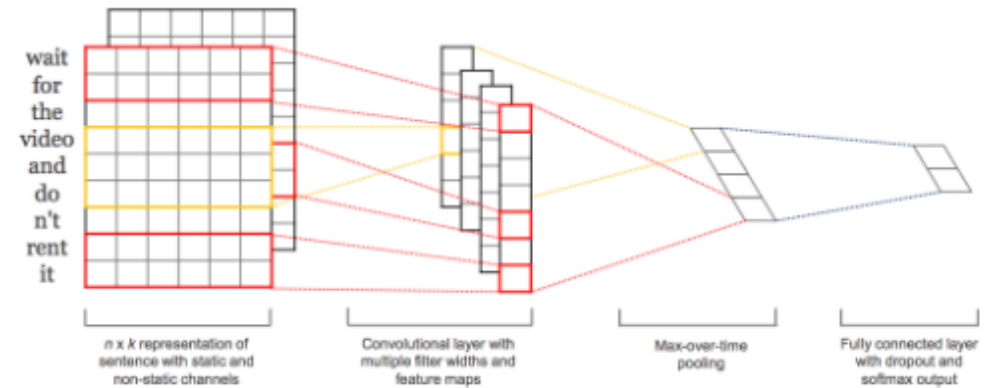


Figure 1: Model architecture with two channels for an example sentence.

Consider: "The movie was not very good."

Small Kernels

A small kernel may detect tightly coupled words:

not good

Large Kernels

A larger kernel may detect extended context:

not very good

An even larger receptive field can capture:

movie was not very good

Therefore: Different kernels provide different levels of local context.

Global Max Pooling Revisited



Feature Map

Suppose a feature map is:

```
[0.1, 0.7, 0.2, 1.4, 0.3, 0.5]
```



Global Max Pooling

Extracts the single largest value:

```
max = 1.4
```

Interpretation:

"The strongest evidence detected by this filter anywhere in the sentence."

Why Global Max Pooling?

- > **Position invariance**
The important feature can occur anywhere in the text.
- > **Dimensionality reduction**
Variable-length feature maps become fixed-size vectors.
- > **Simplicity**
No recurrent processing is required.
- > **Useful for classification**
The classifier mainly needs to know: *Did this important pattern occur?*

Max Pooling vs Average Pooling

Pooling Type	Formula	What it Captures
Max Pooling	$\max (x_1, x_2, \dots, x_n)$	The strongest activation.
Average Pooling	$\frac{1}{n} \sum_i x_i$	The overall activation level.

For sentiment classification, **max pooling** is often intuitive:

"Was a strongly negative phrase detected?"

The Concept

Instead of keeping only the single largest activation:

$\text{TopK}(x)$

we can retain the strongest activations.

Example (k=3)

Feature map:

```
0.2  1.1  0.4  0.9  1.5
      0.3
```

Top-3:

```
1.5  1.1  0.9
```

Advantage:

Retains more information than a single maximum.

Multi-Channel CNN

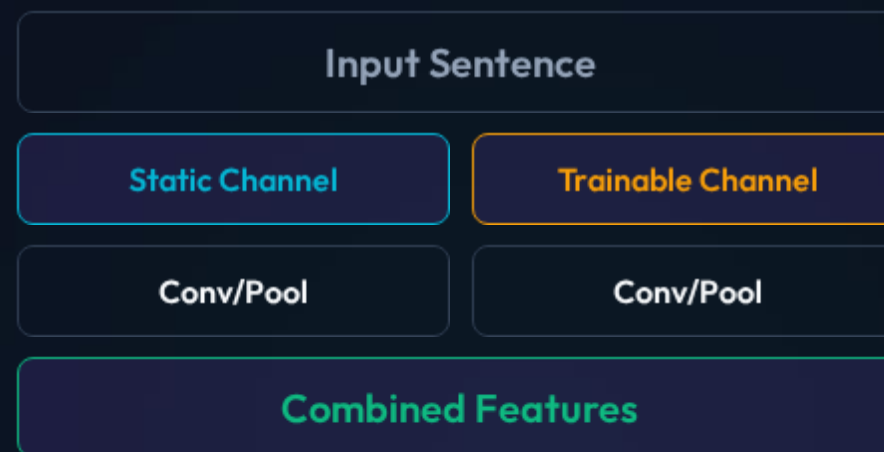
A word can have different representations.

> **Channel 1:**
Static pretrained embeddings (e.g., Word2Vec)

> **Channel 2:**
Trainable embeddings

Both channels receive the same sentence.

Their feature maps can later be combined (concatenated)
before classification.



Static vs Trainable Embeddings

Static Embeddings

Embedding vectors remain fixed during training.

Advantages:

- > Preserves pretrained semantic information
- > Fewer trainable parameters (faster, less overfitting)

Trainable Embeddings

Embedding vectors are updated during training via backpropagation.

Advantages:

- > Adapt specifically to the target task
- > Learn domain-specific representations

Multi-channel approach: Use both.

CNN Architecture for Sentiment Analysis





Complete Mathematical View

Embedding: $X = E(\text{tokens})$

For filter W_k : $c_i = f(W_k \cdot X_{i:i+h-1} + b)$

Feature map: $C_k = [c_1, c_2, \dots, c_n]$

Global max pool: $p_k = \max(C_k)$

For m filters: $p = [p_1, p_2, \dots, p_m]$

Classifier: $\hat{y} = \text{softmax}(W_c p + b_c)$

CNN Hyperparameters

Important design choices and common values:

Hyperparameter	Typical Choice
Embedding dimension	100–300
Number of filters	50–300
Kernel sizes	2, 3, 4, 5
Stride	1
Padding	Valid / Same
Pooling	Global max
Dropout	0.2–0.5
Optimizer	Adam

There is no universally optimal configuration.

CNN Training Pipeline

Data Preparation

- 1 Prepare labeled text
- 2 Tokenize
- 3 Convert tokens to IDs

Forward Pass

- 4 Create embeddings
- 5 Apply convolution
- 6 Apply activation
- 7 Pool
- 8 Classify

Learning

- 9 Calculate loss
- 10 Backpropagate & update parameters

Minimal PyTorch Architecture

```
class TextCNN(nn.Module):  
    def __init__(self, vocab_size, embed_dim):  
        super().__init__()  
        self.embedding = nn.Embedding(vocab_size, embed_dim)  
        self.conv3 = nn.Conv1d(embed_dim, 100, kernel_size=3)  
        self.conv5 = nn.Conv1d(embed_dim, 100, kernel_size=5)  
        self.fc = nn.Linear(200, 2)
```

Embedding → Conv → Pool → Concatenate → Classifier

CNN: Strengths and Limitations

↗Strengths

- > Fast training
- > Parallel computation
- > Effective local feature extraction
- > Good for sentence classification
- > Fewer sequential dependencies
- > Multiple kernel sizes capture different patterns

↘Limitations

- > Limited long-range context
- > Fixed local receptive fields
- > Pooling can discard information
- > Architecture requires tuning
- > Less suitable for complex sequence generation

Summary & Next Lecture



Today We Learned

Padding

Stride

Output-size calc.

Multiple filters

Multiple feature maps

Multiple kernel sizes

Global max pooling

Top-k pooling

Multi-channel CNNs

Static vs trainable

Architecture design

Hyperparameters



Next Lecture

Neural NLP Applications

- > Sentiment Analysis
- > Text Classification
- > Word Sense Disambiguation

Image Sources

600 × 400

http://googleusercontent.com/image_collection/image_retrieval/2059202796781891513_0

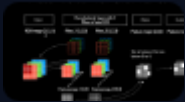
Source: medium.com

600 × 400

http://googleusercontent.com/image_collection/image_retrieval/17539518824136561345_0

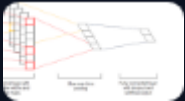
Source: cezannec.github.io

Image Sources



https://miro.medium.com/v2/resize:fit:2000/1*QLzgMSClaZFZUrrGwild_w.png

Source: medium.com



https://cezannec.github.io/assets/cnn_text/complete_text_classification_CNN.png

Source: cezannec.github.io